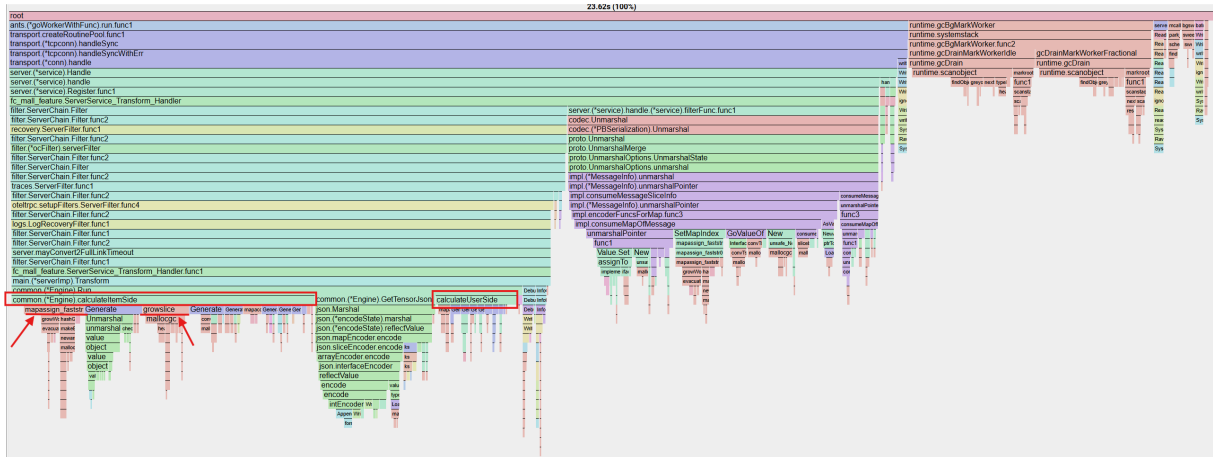


20250728日报

今日工作内容

1. 研究fc_mall_feature现网环境中的火焰图，分析cpu占比大的代码部分；



从火焰图中可知，CalculateItemSide和CalculateUserSide中的map的写操作和slice的扩容占了不小的cpu使用，因此可以通过预分配手段减少其使用率，修改后的代码如下：

```

1  func (e *Engine) modifiedCalculateItemSide(
2      ctx context.Context, itemFeatures []*fc_mall_feature.ItemFeature,
3      userFeature *fc_mall_feature.UserFeature, userSideInner map[string]interface{},
4  ) map[string][]interface{} {
5      result := make(map[string][]interface{})
6      userData := userFeature.Data
7      for _, itemFeature := range itemFeatures {
8          itemData := itemFeature.Data
9          inner := make(map[string]any)
10         for _, def := range e.ItemSide {
11             // 预分配
12             inputs := make([]any, 0, len(def.Inputs)+len(def.InnerInputs))
13             for _, input := range def.Inputs {
14                 if v, ok := itemData[input]; ok {
15                     inputs = append(inputs, v)
16                 } else {
17                     inputs = append(inputs, userData[input])
18                 }
19             }
20             for _, input := range def.InnerInputs {
21                 if v, ok := inner[input]; ok {
22                     inputs = append(inputs, v)
23                 } else {
24                     inputs = append(inputs, userSideInner[input])
25                 }
26             }
27             kvList, err := def.Generator.Generate(def.Name, inputs)
28             if err != nil {
29                 log.Errorf(ctx, "calculate item side %v error: %v",
30                     def.Name, err)
31             }
32             for _, kv := range kvList {
33                 inner[kv.Key] = kv.Value

```

此修改还没有测试，没有比较和之前未修改的性能差异。

2. 在之前的MR后置脚本中添加自动生成__init__.py功能，给diff得到的每个目录都加上__init__.py，修改后的脚本如下：

```
1  source ~/.bashrc
2
3  cd $WORKSPACE
4  echo ${BK_CI_HOOK_TARGET_BRANCH}
5
6  git fetch origin ${BK_CI_HOOK_TARGET_BRANCH}
7  git checkout ${BK_CI_HOOK_TARGET_BRANCH}
8
9  create_init_py() {
10     local dir="$1"
11     init_file="${dir}/__init__.py"
12     if [ ! -f "$init_file" ]; then
13         touch "$init_file"
14         echo "Created: $init_file"
15     fi
16 }
17
18 git diff --name-only HEAD^ HEAD | while read file; do
19     dirname "$file"
20 done | sort -u | grep -v "^. $" | while read dir; do
21     echo "$dir"
22     make dir="$dir"
23     create_init_py "$dir"
24 done
25
26 git add .
27 if ! git diff-index --quiet HEAD --; then
28     git commit -m "auto-build"
29     git push origin ${BK_CI_HOOK_TARGET_BRANCH}
30 else
31     echo "no files changed, skip commit"
32 fi
33
```

自测结果：

```
Bash
1 master
2 来自 http://git.woa.com/drewjjli/helloworld
3 * branch master -> FETCH_HEAD
4 已检出于 'master'
5 您的分支与上游分支 'origin/master' 一致。
6 sgamproto/ai_play_model2
7 Created: sgamproto/ai_play_model2/_init_.py
8 Created: sgamproto/ai_play_model2/lobby_require_invite/_init_.py
9 sgamproto/ai_play_model2/lobby_require_invite
10 [master 8526a97] auto-build
11 2 files changed, 0 insertions(+), 0 deletions(-)
12 create mode 100644 sgamproto/ai_play_model2/_init_.py
13 create mode 100644 sgamproto/ai_play_model2/lobby_require_invite/_init_.py
14 remote:
15 remote: Processing pre-receive: permission-checks: 1remote: Processing pre-receive: event-create: 100% (1/1)remote: Processing pre-receive: watch-path-check: 1remote: Processing pre-receive: push-check: 2remote: Processing pre-receive: push-check: 2 (\\)r
16 remote: Processing pre-receive: push-check: 2, done
17 remote:
18 remote: | To create a merge request for master, visit:
19 remote: | https://git.woa.com/drewjjli/helloworld/-/merge_requests/new?merge_request%5Bsource_branch%5D=master
20 remote:
21 To http://git.woa.com/drewjjli/helloworld.git
22 ca3c47..8526a97 master -> master
23 Latest tag: v0.0.20
24 New tag: v0.0.21
25 remote:
26 remote: Processing pre-receive: permission-checks: 1remote: Processing pre-receive: event-create: 100% (1/1)remote: Processing pre-receive: push-check: 2remote: Processing pre-receive: push-check: 2 (\\)remote: Processing pre-receive: push-check: 2, done
27 To http://git.woa.com/drewjjli/helloworld.git
28 v0.0.21 -> v0.0.21
```

文件名	最后更新于	最后的提交
..		
lobby_require_invite	不到 1 分钟 以前	auto-build
__init__.py	不到 1 分钟 以前	auto-build
ai_play_model.pb.go	不到 1 分钟 以前	auto-build
ai_play_model.proto	2 分钟 以前	test
ai_play_model_pb2.py	不到 1 分钟 以前	auto-build
pb.py	2 分钟 以前	test
rpc.py	2 分钟 以前	test
rpc_stream.py	2 分钟 以前	test

文件名	最后更新于
__init__.py	1 分钟 以前
lobby_require_invite.pb.go	1 分钟 以前
lobby_require_invite.proto	3 分钟 以前
lobby_require_invite_pb2.py	1 分钟 以前

3. 进一步在脚本中添加生成桩代码的功能，脚本如下：

```
1  source ~/.bashrc
2
3  cd $WORKSPACE
4  echo ${BK_CI_HOOK_TARGET_BRANCH}
5
6  git fetch origin ${BK_CI_HOOK_TARGET_BRANCH}
7  git checkout ${BK_CI_HOOK_TARGET_BRANCH}
8
9  create_init_py() {
10     local dir="$1"
11     init_file="${dir}/__init__.py"
12     if [ ! -f "$init_file" ]; then
13         touch "$init_file"
14         echo "Created: $init_file"
15     fi
16 }
17
18 git diff --name-only HEAD^ HEAD | while read file; do
19     dirname "$file"
20 done | sort -u | grep -v "^.$" | while read dir; do
21     echo "$dir"
22     make dir="$dir"
23     create_init_py "$dir"
24     if find "$dir" -maxdepth 1 -name "*.proto" | grep -q .; then
25         echo "Found .proto files in $dir, running make trpc_go"
26         file_name="$proto_file"
27         output_dir=$(dirname "$proto_file" .proto)
28         trpc create --rpconly \
29             --nogomod \
30             --protofile="$proto_file" \
31             --output="$output_dir"
32     else
33         echo "No .proto files found in $dir"
```

由于流水线对应的镜像没有安装trpc-go环境，因此还没有运行成功这个脚本，现在正在给镜像配置trpc-go环境。

明日待办：

1. 把镜像拉下来后安装trpc-go环境，并在流水线中测试添加了生成桩代码后的脚本；
2. 继续研究fc_mall_feature性能优化；